

Trabajo Práctico Integrador

Juego por turnos tipo Piedra–Papel–Tijera (Agua, Tierra y Fuego)

Grupos no más de 3 y menos de 2 personas.

Entrega **28/09/2026**

La entrega se deberá realizar en un archivo comprimido el cual deberá ser subido al campus, solo por uno de los integrantes del grupo.

1. Introducción

El presente Trabajo Práctico tiene como objetivo aplicar los principios de Programación Orientada a Objetos en el diseño e implementación de un juego por turnos basado en una mecánica tipo Piedra–Papel–Tijera, utilizando elementos:

-  Agua
-  Tierra
-  Fuego

Cada elemento posee ventajas y desventajas frente a los demás, generando interacciones estratégicas durante el combate.

A diferencia del juego clásico, cada jugador dispondrá de un conjunto de elementos (cartas/unidades), los cuales tendrán un nivel de energía (vida). Durante cada turno, los elementos se enfrentan y se aplican daños configurables según la combinación de tipos.

El objetivo del juego es reducir la energía del oponente hasta dejarlo sin unidades activas.

El sistema deberá ser flexible, permitiendo modificar reglas de daño sin cambiar la lógica principal del juego.

2. Descripción General del Sistema

El sistema a diseñar y desarrollar deberá:

- Permitir definir una partida entre jugador y computadora.
- Generar un conjunto de N elementos (por ejemplo 6) de manera aleatoria.
- Asignar a cada elemento:
 - Tipo (Agua, Tierra o Fuego)
 - Energía inicial (100%)
- Implementar un sistema de combate por turnos.
- Aplicar daño según una matriz configurable de efectividad entre tipos.
- Determinar el ganador en función de la energía restante.

Mecánica del Juego

- Cada jugador posee un conjunto de elementos.
- En cada turno:
 - Se selecciona un elemento propio.
 - Se enfrenta contra un elemento del oponente.
- Se aplica daño según el tipo enfrentado.

Ejemplo de configuración de daño

Atacante	Defensor	Daño
Agua	Fuego	50%
Tierra	Fuego	20%
Fuego	Tierra	40%
Agua	Tierra	30%

etc.

- La energía se reduce en cada enfrentamiento.
- Cuando un elemento llega a 0%, queda fuera de combate.
- Gana el jugador que:
 - Mantiene al menos un elemento con energía, y
 - Deja al oponente sin elementos activos.

Descripción:

Siempre participan dos jugadores: un jugador humano y una inteligencia artificial (IA).

Al comenzar la partida, cada jugador recibe 5 elementos de forma aleatoria (por ejemplo: agua, tierra, fuego, etc.). Cada elemento comienza con un 100% de vida.

El juego se desarrolla en rondas. En cada ronda, ambos jugadores seleccionan uno de sus elementos disponibles para combatir.

Cuando dos elementos se enfrentan, cada uno realiza un ataque que reduce la vida del oponente según una regla de efectividad entre elementos (por ejemplo: agua le quita 50% a fuego, mientras que fuego le quita 20% a agua).

Si la vida de un elemento llega a 0%, este muere y debe ser reemplazado por otro elemento disponible del mismo jugador.

El jugador humano puede elegir qué elemento usar en cada turno, mientras que la IA selecciona uno automáticamente.

El juego continúa de esta manera hasta que uno de los jugadores se queda sin elementos vivos. En ese momento, el otro jugador es declarado ganador.

Ejemplo de partida

Siempre hay 2 jugadores: uno humano y otro máquina (IA).

Al comenzar la partida, a cada jugador se le asignan 5 elementos de forma aleatoria.

Por ejemplo:

- Jugador humano: agua, tierra, tierra, fuego, fuego
- IA: fuego, agua, tierra, fuego, tierra

Todos los elementos comienzan con 100% de vida.

El jugador humano elige un elemento para comenzar, por ejemplo, agua.

La IA también elige uno, por ejemplo, fuego.

Primera ronda:

El agua ataca primero y le quita un 50% de vida al fuego.

Luego, el fuego ataca y le quita un 20% al agua.

Segunda ronda:

El agua vuelve a atacar y elimina al fuego (llevando su vida a 0%).

La IA debe entonces seleccionar un nuevo elemento, por ejemplo, tierra.

El ataque de tierra reduce la vida del agua en un 50%, dejándola con un 30% de vida.

Tercera ronda:

El agua ataca a tierra y le quita un 20% de vida.

Luego, tierra contraataca y elimina al agua.

El jugador humano debe elegir entonces otro elemento de su lista inicial.

El juego continúa de esta manera, enfrentando elementos en rondas sucesivas, hasta que uno de los jugadores se queda sin elementos con vida. En ese momento, su oponente es declarado ganador.

Tipos de IA

El juego contempla distintos tipos de inteligencia artificial para controlar al oponente. Cada tipo define una estrategia diferente y estas pueden tocarse de forma aleatoria.

IA Aleatoria:

Selecciona un elemento al azar entre los disponibles, sin tener en cuenta el elemento del oponente ni posibles ventajas o desventajas.

IA Estratégica:

Selecciona el elemento que le otorgue mayor ventaja frente al elemento del oponente. Por ejemplo, si el jugador utiliza agua, la IA elegirá tierra para maximizar el daño infligido y minimizar el recibido.

Super IA (estrategia + eficiencia) :

Selecciona el elemento que le otorgue mayor ventaja frente al elemento del oponente. Por ejemplo, si el jugador utiliza agua, la IA elegirá tierra para maximizar el daño infligido y minimizar el recibido. Pero, tratando de maximizar los beneficios, por ejemplo, al humano le queda un elemento de agua al 20%; puede elegir cualquiera que el ataque quite 20% o más.

Objetivo

El objetivo principal de esta etapa es validar el modelo propuesto y comenzar a materializarlo en código, manteniendo los principios de Programación Orientada a Objetos definidos en el diseño original.

Implementación Inicial del Sistema

Se comenzará con la implementación de los componentes principales del sistema, permitiendo:

- Crear elementos (Agua, Tierra y Fuego).
- Crear jugadores.
- Asignar elementos aleatoriamente.
- Administrar energía de los elementos.
- Implementar el sistema de daño configurable.
- Realizar enfrentamientos entre dos elementos.
- Detectar cuando un elemento queda fuera de combate.
- Mostrar el estado actual de la partida.
- Debe ser posible iniciar una partida y ejecutar combates entre elementos.

Criterios de Evaluación

- Correcta aplicación de los conceptos de POO.
- Funcionalidad del software de acuerdo con las reglas y requerimientos establecidos.

Consideraciones

- No se permite lógica centralizada en una sola clase.
- No se permite resolver interacciones con múltiples `if` o `switch`.
- No se permite el uso de estructuras sin encapsulación.
- No se permite resolver el sistema como funciones globales.